Softvérové jazyky
Jazyk SMALL

Lukáš Častven, Michal Kilian

Apríl 2026

1 BNF

Jazyk SMALL je jednoduchý programovací jazyk. Obsahuje sekvencie príkazov, priradenia, vstup a výstup, podmienený príkaz `IF`, aritmetické výrazy a booleovské výrazy s operátormi `OR`, `AND` a `NOT`.

1. `program` := `"BEGIN"` `statement_list` `"END"`
2. `statement_list` := `statement` {`statement`}
3. `statement` := `ident` `":="` `expression` `";"`
4. `statement` := `"READ"` `"("` `id_list` `")"` `";"`
5. `statement` := `"WRITE"` `"("` `expr_list` `")"` `";"`
6. `statement` := `"IF"` `bexpr` `"THEN"` `statement` [`"ELSE"` `statement`] `";"`
7. `id_list` := `ident` {`","` `ident`}
8. `expr_list` := `expression` {`","` `expression`}
9. `expression` := [`expression` `op`] `factor`
10. `factor` := `"("` `expression` `")"`
11. `factor` := `ident` | `number`
12. `op` := `"+"` | `"-"`
13. `bexpr` := [`bexpr` `"OR"`] `bterm`
14. `bterm` := [`bterm` `"AND"`] `bfactor`
15. `bfactor` := `"NOT"` `bfactor` | `"("` `bexp` `")"` | `"TRUE"` | `"FALSE"`
16. `ident` := `letter` {`letter` | `digit09`}
17. `letter` := `"a"` | ... | `"z"` | `"A"` | ... | `"Z"`
18. `number` := [`"+"` | `"-"`] `digit19` {`digit09`}
19. `digit09` := `"0"` | ... | `"9"`
20. `digit19` := `"1"` | ... | `"9"`

2 Transformácia BNF

Pri prepisovaní BNF do gramatiky boli použité tieto pravidlá:

1. $[E]$ sa transformuje na $E \mid \varepsilon$
2. $(E \mid F)G$ sa transformuje na $EG \mid FG$
3. $E := F\{aF\}$ sa transformuje na $E \to F \mid FaE$
4. $E := F\{aG\}$ sa transformuje na $E \to FX$, $X \to aGX \mid \varepsilon$

Na základe týchto pravidiel bola BNF prepísaná do nasledujúcej podoby [1]:

- `program` → `"BEGIN"` `statement_list` `"END"`
- `statement_list` → `statement` | `statement` `statement_list` (použité pravidlo 3)
- `statement` → `ident` `":="` `expression` `";"`
- `statement` → `"READ"` `"("` `id_list` `")"` `";"`
- `statement` → `"WRITE"` `"("` `expr_list` `")"` `";"`
- `statement` → `"IF"` `bexpr` `"THEN"` `statement` `else` `";"`
- `else` → ε | `"ELSE"` `statement` (použité pravidlo 1)
- `id_list` → `ident` | `ident` `","` `id_list` (použité pravidlo 3)
- `expr_list` → `expression` | `expression` `","` `expr_list` (použité pravidlo 3)
- `expression` → `factor` | `expression` `op` `factor` (použité pravidlo 1)
- `factor` → `"("` `expression` `")"`
- `factor` → `ident` | `number`
- `op` → `"+"` | `"-"`
- `bexpr` → `bterm` | `bexpr` `"OR"` `bterm` (použité pravidlo 1)
- `bterm` → `bfactor` | `bterm` `"AND"` `bfactor` (použité pravidlo 1)
- `bfactor` → `"NOT"` `bfactor` | `"("` `bexpr` `")"` | `"TRUE"` | `"FALSE"`
- `ident` → `letter` {`letter` | `digit09`}
- `dent` → ε | `letter` `dent` | `digit09` `dent` (použité pravidlo 4)
- `letter` → `"a"` | ... | `"z"` | `"A"` | ... | `"Z"`
- `number` → `sign` `digit19` {`digit09`}
- `umber` → ε | `digit09` `umber` (použité pravidlo 1)
- `sign` → ε | `"+"` | `"-"` (použité pravidlo 1)
- `digit09` → `"0"` | ... | `"9"`
- `digit19` → `"1"` | ... | `"9"`

3 Odstránenie ľavej rekurzie

Ľavá rekurzia sa odstraňovala preto, aby bolo možné gramatiku dostať do `LL(1)` tvaru. Iba pravidlá `expression`, `bexpr` a `bterm` boli upravené [2]:

- `expression` → `factor` `expression'`
- `expression'` → ε | `op` `factor` `expression'`
- `bexpr` → `bterm` `bexpr'`
- `bexpr'` → `"OR"` `bterm` `bexpr'` | ε
- `bterm` → `bfactor` `bterm'`
- `bterm'` → `"AND"` `bfactor` `bterm'` | ε

[1] Originálne prepísanie je v súbore `docs/2_grammar.md`, sem bolo len prepísané
[2] Originálne prepísanie je v súbore `docs/3_left-rec.md`, sem bolo len prepísané

4 Ľavá faktorizácia

Ľavá faktorizácia sa používala na odstránenie spoločných začiatkov pravidiel. Iba pravidlá `statement_list`, `id_list` a `expr_list` boli upravené [3]:

- `statement_list` → `statement statement_list'`
- `statement_list'` → ε | `statement statement_list'`
- `id_list` → `ident id_list'`
- `id_list'` → ε | `"," id_list`
- `expr_list` → `expression expr_list'`
- `expr_list'` → ε | `"," expr_list`

5 LL(1) gramatika

Pre výslednú gramatiku boli vykonané kontroly konfliktov `F1 x F1` a `FO1 x F1`. Overovalo sa, či sa množiny `FIRST` jednotlivých alternatív neprekrývajú a či sa pri pravidlách s ε neprekrývajú so `FOLLOW` množinou. Všetky kontroly dopadli bez konfliktov, takže výsledná gramatika je `LL(1)` [4].

1. `program` → `"BEGIN" statement_list "END"`
   (a) $F1(\texttt{program}) = \{\texttt{BEGIN}\}$
   (b) $FO1(\texttt{program}) = \{\$\}$
   (c) *bez konfliktu, iba jedno pravidlo*
2. `statement_list` → `statement statement_list'`
   (a) $F1(\texttt{statement_list}) = F1(\texttt{statement statement_list'}) = \{\texttt{ident}, \texttt{READ}, \texttt{WRITE}, \texttt{IF}\}$
   (b) $FO1(\texttt{statement_list}) = F1(\texttt{END}) = \{\texttt{END}\}$
   (c) *bez konfliktu, iba jedno pravidlo*
3. `statement_list'` → ε | `statement_list`
   (a) $F1(\texttt{statement_list'}) = F1(\varepsilon) \cup F1(\texttt{statement_list}) = \{\texttt{ident}, \texttt{READ}, \texttt{WRITE}, \texttt{IF}, \varepsilon\}$
   (b) i. $F1(\varepsilon) = \{\varepsilon\}$
   ii. $F1(\texttt{statement_list}) = \{\texttt{ident}, \texttt{READ}, \texttt{WRITE}, \texttt{IF}\}$
   (c) $FO1(\texttt{statement_list'}) = \{\texttt{END}\}$
   (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*
   (e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*
4. `statement` → `ident ":=" expression ";"` | `"READ" "(" id_list ")" ";"` | `"WRITE" "(" expr_list ")" ";"` | `"IF" bexpr "THE`
   (a) $F1(\texttt{statement}) = \{\texttt{ident}, \texttt{READ}, \texttt{WRITE}, \texttt{IF}\}$
   (b) i. $F1(\texttt{ident} := \texttt{expression};) = \{\texttt{ident}\}$
   ii. $F1(\texttt{READ}(\texttt{id_list});) = \{\texttt{READ}\}$
   iii. $F1(\texttt{WRITE}(\texttt{expr_list});) = \{\texttt{WRITE}\}$
   iv. $F1(\texttt{IF bexpr THEN statement else};) = \{\texttt{IF}\}$
   (c) $FO1(\texttt{statement}) = \{\texttt{ident}, \texttt{READ}, \texttt{WRITE}, \texttt{IF}, \texttt{END}, \texttt{ELSE}, \texttt{;}\}$
   (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*
5. `else` → ε | `ELSE statement`
   (a) $F1(\texttt{else}) = F1(\varepsilon) \cup F1(\texttt{ELSE statement}) = \{\texttt{ELSE}, \varepsilon\}$
   (b) $FO1(\texttt{else}) = \{\texttt{;}\}$
   (c) i. $F1(\varepsilon) = \{\varepsilon\}$

[3] Originálne prepísanie je v súbore `docs/4_left-fact.md`, sem bolo len prepísané
[4] Originálne vyhodnotenie či je gramaticka LL(1) je v súbore `docs/5_ll1.md`, sem bolo len prepísané

ii. $F1(\texttt{ELSE statement}) = \{\texttt{ELSE}\}$

(d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*

(e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*

6. $\texttt{id_list} \rightarrow \texttt{ident id_list'}$

   (a) $F1(\texttt{id_list}) = F1(\texttt{ident id_list'}) = \{\texttt{ident}\}$

   (b) $FO1(\texttt{id_list}) = \{\texttt{)}\}$

   (c) *bez konfliktu, iba jedno pravidlo*

7. $\texttt{id_list'} \rightarrow \varepsilon \mid \texttt{, id_list}$

   (a) $F1(\texttt{id_list'}) = F1(\varepsilon) \cup F1(\texttt{, id_list}) = \{\texttt{,}, \varepsilon\}$

   (b) i. $F1(\varepsilon) = \{\varepsilon\}$

   ii. $F1(\texttt{, id_list}) = \{\texttt{,}\}$

   (c) $FO1(\texttt{id_list'}) = \{\texttt{)}\}$

   (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*

   (e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*

8. $\texttt{expr_list} \rightarrow \texttt{expression expr_list'}$

   (a) $F1(\texttt{expr_list}) = F1(\texttt{expression expr_list'}) = \{\texttt{ident}, \texttt{+}, \texttt{-}, \texttt{1}, \ldots, \texttt{9}, \texttt{(}\}$

   (b) $FO1(\texttt{expr_list}) = \{\texttt{)}\}$

   (c) *bez konfliktu, iba jedno pravidlo*

9. $\texttt{expr_list'} \rightarrow \varepsilon \mid \texttt{, expr_list}$

   (a) $F1(\texttt{expr_list'}) = F1(\varepsilon) \cup F1(\texttt{, expr_list}) = \{\texttt{,}, \varepsilon\}$

   (b) i. $F1(\varepsilon) = \{\varepsilon\}$

   ii. $F1(\texttt{, expr_list}) = \{\texttt{,}\}$

   (c) $FO1(\texttt{expr_list'}) = \{\texttt{)}\}$

   (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*

   (e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*

10. $\texttt{expression} \rightarrow \texttt{factor expression'}$

    (a) $F1(\texttt{expression}) = F1(\texttt{factor expression'}) = \{\texttt{ident}, \texttt{+}, \texttt{-}, \texttt{1}, \ldots, \texttt{9}, \texttt{(}\}$

    (b) $FO1(\texttt{expression}) = \{\texttt{;}, \texttt{,}, \texttt{)}\}$

    (c) *bez konfliktu, iba jedno pravidlo*

11. $\texttt{expression'} \rightarrow \varepsilon \mid \texttt{op factor expression'}$

    (a) $F1(\texttt{expression'}) = F1(\varepsilon) \cup F1(\texttt{op factor expression'}) = \{\texttt{+}, \texttt{-}, \varepsilon\}$

    (b) i. $F1(\varepsilon) = \{\varepsilon\}$

    ii. $F1(\texttt{op factor expression'}) = \{\texttt{+}, \texttt{-}\}$

    (c) $FO1(\texttt{expression'}) = \{\texttt{;}, \texttt{,}, \texttt{)}\}$

    (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*

    (e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*

12. $\texttt{factor} \rightarrow \texttt{ident} \mid \texttt{number} \mid \texttt{(expression)}$

    (a) $F1(\texttt{factor}) = F1(\texttt{ident}) \cup F1(\texttt{number}) \cup F1(\texttt{(expression)}) = \{\texttt{ident}, \texttt{+}, \texttt{-}, \texttt{1}, \ldots, \texttt{9}, \texttt{(}\}$

    (b) i. $F1(\texttt{ident}) = \{\texttt{ident}\}$

    ii. $F1(\texttt{number}) = \{\texttt{+}, \texttt{-}, \texttt{1}, \ldots, \texttt{9}\}$

    iii. $F1(\texttt{(expression)}) = \{\texttt{(}\}$

    (c) $FO1(\texttt{factor}) = \{\texttt{+}, \texttt{-}, \texttt{;}, \texttt{,}, \texttt{)}\}$

    (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*

13. $\texttt{op} \rightarrow + \mid -$
    (a) $F1(\texttt{op}) = \{\texttt{+}, \texttt{-}\}$
    (b) $FO1(\texttt{op}) = \{\texttt{ident}, \texttt{number}, \texttt{(}\}$
    (c) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*
14. $\texttt{bexpr} \rightarrow \texttt{bterm}\,\texttt{bexpr'}$
    (a) $F1(\texttt{bexpr}) = F1(\texttt{bterm}\,\texttt{bexpr'}) = \{\texttt{NOT}, \texttt{(}, \texttt{TRUE}, \texttt{FALSE}\}$
    (b) $FO1(\texttt{bexpr}) = \{\texttt{THEN}, \texttt{)}\}$
    (c) *bez konfliktu, iba jedno pravidlo*
15. $\texttt{bexpr'} \rightarrow \varepsilon \mid \texttt{OR}\,\texttt{bterm}\,\texttt{bexpr'}$
    (a) $F1(\texttt{bexpr'}) = F1(\varepsilon) \cup F1(\texttt{OR}\,\texttt{bterm}\,\texttt{bexpr'}) = \{\texttt{OR}, \varepsilon\}$
    (b) i. $F1(\varepsilon) = \{\varepsilon\}$
        ii. $F1(\texttt{OR}\,\texttt{bterm}\,\texttt{bexpr'}) = \{\texttt{OR}\}$
    (c) $FO1(\texttt{bexpr'}) = \{\texttt{THEN}, \texttt{)}\}$
    (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*
    (e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*
16. $\texttt{bterm} \rightarrow \texttt{bfactor}\,\texttt{bterm'}$
    (a) $F1(\texttt{bterm}) = F1(\texttt{bfactor}\,\texttt{bterm'}) = \{\texttt{NOT}, \texttt{(}, \texttt{TRUE}, \texttt{FALSE}\}$
    (b) $FO1(\texttt{bterm}) = \{\texttt{OR}, \texttt{THEN}, \texttt{)}\}$
    (c) *bez konfliktu, iba jedno pravidlo*
17. $\texttt{bterm'} \rightarrow \varepsilon \mid \texttt{AND}\,\texttt{bfactor}\,\texttt{bterm'}$
    (a) $F1(\texttt{bterm'}) = F1(\varepsilon) \cup F1(\texttt{AND}\,\texttt{bfactor}\,\texttt{bterm'}) = \{\texttt{AND}, \varepsilon\}$
    (b) i. $F1(\varepsilon) = \{\varepsilon\}$
        ii. $F1(\texttt{AND}\,\texttt{bfactor}\,\texttt{bterm'}) = \{\texttt{AND}\}$
    (c) $FO1(\texttt{bterm'}) = \{\texttt{OR}, \texttt{THEN}, \texttt{)}\}$
    (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*
    (e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*
18. $\texttt{bfactor} \rightarrow \texttt{NOT}\,\texttt{bfactor} \mid \texttt{(}\,\texttt{bexpr}\,\texttt{)} \mid \texttt{TRUE} \mid \texttt{FALSE}$
    (a) $F1(\texttt{bfactor}) = F1(\texttt{NOT}\,\texttt{bfactor}) \cup F1(\texttt{(}\,\texttt{bexpr}\,\texttt{)}) \cup F1(\texttt{TRUE}) \cup F1(\texttt{FALSE}) = \{\texttt{NOT}, \texttt{(}, \texttt{TRUE}, \texttt{FALSE}\}$
    (b) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*
19. $\texttt{ident} \rightarrow \texttt{letter}\,\texttt{dent}$
    (a) $F1(\texttt{ident}) = F1(\texttt{letter}) = \{\texttt{a}, \ldots, \texttt{z}, \texttt{A}, \ldots, \texttt{Z}\}$
    (b) $FO1(\texttt{ident}) = \{\texttt{:=}, \texttt{)}, \texttt{+}, \texttt{-}, \texttt{;}\}$
    (c) *bez konfliktu, iba jedno pravidlo*
20. $\texttt{dent} \rightarrow \varepsilon \mid \texttt{letter}\,\texttt{dent} \mid \texttt{digit09}\,\texttt{dent}$
    (a) $F1(\texttt{dent}) = F1(\varepsilon) \cup F1(\texttt{letter}\,\texttt{dent}) \cup F1(\texttt{digit09}\,\texttt{dent}) = \{\texttt{a}, \ldots, \texttt{z}, \texttt{A}, \ldots, \texttt{Z}, \texttt{0}, \ldots, \texttt{9}, \varepsilon\}$
    (b) i. $F1(\varepsilon) = \{\varepsilon\}$
        ii. $F1(\texttt{letter}\,\texttt{dent}) = \{\texttt{a}, \ldots, \texttt{z}, \texttt{A}, \ldots, \texttt{Z}\}$
        iii. $F1(\texttt{digit09}\,\texttt{dent}) = \{\texttt{0}, \ldots, \texttt{9}\}$
    (c) $FO1(\texttt{dent}) = \{\texttt{:=}, \texttt{)}, \texttt{+}, \texttt{-}, \texttt{;}\}$
    (d) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*
    (e) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*
21. $\texttt{letter} \rightarrow \texttt{a} \mid \ldots \mid \texttt{z} \mid \texttt{A} \mid \ldots \mid \texttt{Z}$
    (a) $F1(\texttt{letter}) = \{\texttt{a}, \ldots, \texttt{z}, \texttt{A}, \ldots, \texttt{Z}\}$

(b) $FO1(\texttt{letter}) = \{\texttt{0}, \ldots, \texttt{9}, \texttt{:=}, \texttt{)}, \texttt{+}, \texttt{-}, \texttt{;}\}$

(c) *bez konfliktu, bez prieniku* `FIRST` *medzi alternatívami*

22. $\texttt{number} \rightarrow \texttt{sign}\,\texttt{digit19}\,\texttt{umber}$

    (a) $F1(\texttt{number}) = F1(\texttt{sign}\,\texttt{digit19}\,\texttt{umber}) = F1(\texttt{sign})/\varepsilon \cup F1(\texttt{digit19}) = \{\texttt{+}, \texttt{-}, \texttt{1}, \ldots, \texttt{9}\}$

    (b) $FO1(\texttt{number}) = \{\texttt{+}, \texttt{-}, \texttt{;}, \texttt{,}, \texttt{)}\}$

    (c) *bez konfliktu, iba jedno pravidlo*

23. $\texttt{umber} \rightarrow \varepsilon \mid \texttt{digit09}\,\texttt{umber}$

    (a) $F1(\texttt{umber}) = F1(\texttt{digit09}\,\texttt{umber}) \cup F1(\varepsilon) = \{\texttt{0}, \ldots, \texttt{9}, \varepsilon\}$

    (b) $FO1(\texttt{umber}) = \{\texttt{+}, \texttt{-}, \texttt{;}, \texttt{,}, \texttt{)}\}$

    (c) $F1(\texttt{digit09}\,\texttt{umber}) \cap F1(\varepsilon) = \emptyset$

    (d) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*

24. $\texttt{sign} \rightarrow \varepsilon \mid \texttt{+} \mid \texttt{-}$

    (a) $F1(\texttt{sign}) = \{\texttt{+}, \texttt{-}, \varepsilon\}$

    (b) $FO1(\texttt{sign}) = \{\texttt{1}, \ldots, \texttt{9}\}$

    (c) *bez konfliktu, bez prieniku medzi* `FIRST` *medzi alternatívami*

    (d) *bez konfliktu, bez prieniku medzi* `FIRST` *alternatív a* `FOLLOW` *pravidla*

25. $\texttt{digit09} \rightarrow \texttt{0} \mid \ldots \mid \texttt{9}$

    (a) $F1(\texttt{digit09}) = \{\texttt{0}, \texttt{1}, \ldots, \texttt{9}\}$

    (b) $FO1(\texttt{digit09}) = \{\texttt{a}, \ldots, \texttt{z}, \texttt{A}, \ldots, \texttt{Z}, \texttt{0}, \ldots, \texttt{9}, \texttt{:=}, \texttt{,}, \texttt{)}, \texttt{+}, \texttt{-}, \texttt{;}\}$

    (c) *bez konfliktu, bez prieniku medzi* `FIRST` *medzi alternatívami*

26. $\texttt{digit19} \rightarrow \texttt{1} \mid \ldots \mid \texttt{9}$

    (a) $F1(\texttt{digit19}) = \{\texttt{1}, \ldots, \texttt{9}\}$

    (b) $FO1(\texttt{digit19}) = \{\texttt{0}, \ldots, \texttt{9}, \texttt{+}, \texttt{-}, \texttt{;}, \texttt{,}, \texttt{)}\}$

    (c) *bez konfliktu, bez prieniku medzi* `FIRST` *medzi alternatívami*

6 Tokenizácia

Pri tokenizácii sa niektoré lexikálne pravidlá zmenili na tokeny, aby parser pracoval už len so symbolmi, ktoré mu dodá lexer. Týka sa to pravidiel `ident` a `number`, ktoré sa po tokenizácii nahradili tokenmi **IDENT** a **NUMBER**. Pravidlá pre `dent`, `letter`, `umber`, `sign`, `digit09` a `digit19` sa z gramatiky odstránili, pretože ich spracúva lexer.[5]

1. **AND** = `AND`
2. **BEGIN** = `BEGIN`
3. **END** = `END`
4. **ELSE** = `ELSE`
5. **FALSE** = `FALSE`
6. **IF** = `IF`
7. **NOT** = `NOT`
8. **OR** = `OR`
9. **READ** = `READ`
10. **WRITE** = `WRITE`
11. **THEN** = `THEN`

[5] Originálne prepísanie je v súbore `docs/6_ll1_tokens.md`, sem bolo len prepísané.

12. **TRUE** = `TRUE`
13. **IDENT** = `letter` (`letter` | `digit09`)*
14. **NUMBER** = `sign`? `digit19` `digit09`*
15. **ASSIGN** = `:=`
16. **SEMI** = `;`
17. **LPAREN** = `(`
18. **RPAREN** = `)`
19. **COMMA** = `,`
20. **PLUS** = `+`
21. **MINUS** = `-`

1. `program` → **BEGIN** `statement_list` **END**
   - $F1 = \{$**BEGIN**$\}$
2. `statement_list` → `statement` `statement_list'`
   - $F1 = \{$**IDENT**, **READ**, **WRITE**, **IF**$\}$
3. `statement_list'` → e
   - $FO1 = \{$**END**$\}$
4. `statement_list'` → `statement_list`
   - $F1 = \{$**IDENT**, **READ**, **WRITE**, **IF**$\}$
5. `statement` → **IDENT ASSIGN** `expression` **SEMI**
   - $F1 = \{$**IDENT**$\}$
6. `statement` → **READ LPAREN** `id_list` **RPAREN SEMI**
   - $F1 = \{$**READ**$\}$
7. `statement` → **WRITE LPAREN** `expr_list` **RPAREN SEMI**
   - $F1 = \{$**WRITE**$\}$
8. `statement` → **IF** `bexpr` **THEN** `statement` `else` **SEMI**
   - $F1 = \{$**IF**$\}$
9. `else` → e
   - $FO1 = \{$**SEMI**$\}$
10. `else` → **ELSE** `statement`
    - $F1 = \{$**ELSE**$\}$
11. `id_list` → **IDENT** `id_list'`
    - $F1 = \{$**IDENT**$\}$
12. `id_list'` → e
    - $FO1 = \{$**RPAREN**$\}$
13. `id_list'` → **COMMA** `id_list`
    - $F1 = \{$**COMMA**$\}$
14. `expr_list` → `expression` `expr_list'`

- $F1 = \{\textbf{IDENT}, \textbf{NUMBER}, \textbf{LPAREN}\}$

15. `expr_list'` $\rightarrow$ e
    - $FO1 = \{\textbf{RPAREN}\}$
16. `expr_list'` $\rightarrow$ **COMMA** `expr_list`
    - $F1 = \{\textbf{COMMA}\}$
17. `expression` $\rightarrow$ `factor expression'`
    - $F1 = \{\textbf{IDENT}, \textbf{NUMBER}, \textbf{LPAREN}\}$
18. `expression'` $\rightarrow$ e
    - $FO1 = \{\textbf{SEMI}, \textbf{COMMA}, \textbf{RPAREN}\}$
19. `expression'` $\rightarrow$ `op factor expression'`
    - $F1 = \{\textbf{PLUS}, \textbf{MINUS}\}$
20. `factor` $\rightarrow$ **IDENT**
    - $F1 = \{\textbf{IDENT}\}$
21. `factor` $\rightarrow$ **NUMBER**
    - $F1 = \{\textbf{NUMBER}\}$
22. `factor` $\rightarrow$ **LPAREN** `expression` **RPAREN**
    - $F1 = \{\textbf{LPAREN}\}$
23. `op` $\rightarrow$ **PLUS**
    - $F1 = \{\textbf{PLUS}\}$
24. `op` $\rightarrow$ **MINUS**
    - $F1 = \{\textbf{MINUS}\}$
25. `bexpr` $\rightarrow$ `bterm bexpr'`
    - $F1 = \{\textbf{NOT}, \textbf{LPAREN}, \textbf{TRUE}, \textbf{FALSE}\}$
26. `bexpr'` $\rightarrow$ e
    - $FO1 = \{\textbf{THEN}, \textbf{RPAREN}\}$
27. `bexpr'` $\rightarrow$ **OR** `bterm bexpr'`
    - $F1 = \{\textbf{OR}\}$
28. `bterm` $\rightarrow$ `bfactor bterm'`
    - $F1 = \{\textbf{NOT}, \textbf{LPAREN}, \textbf{TRUE}, \textbf{FALSE}\}$
29. `bterm'` $\rightarrow$ e
    - $FO1 = \{\textbf{OR}, \textbf{THEN}, \textbf{RPAREN}\}$
30. `bterm'` $\rightarrow$ **AND** `bfactor bterm'`
    - $F1 = \{\textbf{AND}\}$
31. `bfactor` $\rightarrow$ **NOT** `bfactor`
    - $F1 = \{\textbf{NOT}\}$
32. `bfactor` $\rightarrow$ **LPAREN** `bexpr` **RPAREN**
    - $F1 = \{\textbf{LPAREN}\}$
33. `bfactor` $\rightarrow$ **TRUE**
    - $F1 = \{\textbf{TRUE}\}$
34. `bfactor` $\rightarrow$ **FALSE**
    - $F1 = \{\textbf{FALSE}\}$

7 Prechodová tabuúlka

Prechodová tabuľka vychádza z tokenizovanej gramatiky. Čísla v bunkách odkazujú na pravidlá z predchádzajúcej sekcie.

Non-terminal	BEGIN	END	READ	WRITE	IF	ELSE	THEN	OR	AND	NOT	TRUE	FALSE	IDENT	NUMBER	ASSIGN	SEMI	LPAREN	RPAREN	COMMA	PLUS	MINUS	$
program	1																					
statement_list			2	2	2								2									
statement_list'		3	4	4	4								4									
statement			6	7	8								5									
else						10										9						
id_list													11									
id_list'																		12	13			
expr_list													14	14			14					
expr_list'																		15	16			
expression													17	17			17					
expression'																18		18	18	19	19	
factor													20	21			22					
op																				23	24	
bexpr										25	25	25					25					
bexpr'							26	27										26				
bterm										28	28	28					28					
bterm'							29	29	30									29				
bfactor										31	33	34					32					

8 Lexikálna analýza

Lexikálna analýza je implementovaná pomocou konečného stavového automatu (FSM), ktorý postupne rozpoznáva kľúčové slová, identifikátory, čísla a symboly. Automat pri spracovaní vstupu prechádza medzi stavmi podľa aktuálneho znaku a po dosiahnutí akceptačného stavu vytvorí príslušný token. Diagram FSM je uložený v súbore `docs/lexer-fsm.png`.

8.1 Zotavenie z chýb

V lexikálnej analýze boli implementované dva režimy zotavenia z chýb.

Prvý režim je preskakovanie znakov dovtedy, kým sa nenájde znak, ktorý umožní pokračovať v rozpoznávaní tokenu. Medzitým sa všetky znaky preskočia a po nájdení správneho znaku sa token emitne. Tento režim sa dá overiť napríklad na tokene `:=`. Ak lexer narazí na `:`, ďalej hľadá `=` a všetko medzi tým preskočí. Podobne pri znaku `0` sa tento znak iba preskočí, pretože `0` nemôže začínať žiadny token a lexer pokračuje v hľadaní ďalšieho vhodného znaku.

Druhý režim je doplnenie chýbajúceho znaku. Tento režim sa dá jednoznačne použiť pri tokene `:=`, kde sa pri chýbajúcom znaku doplní `=`. Pri vstupe začínajúcom na `0` sa tento znak nahradí znakom `R`, aby sa z nejednoznačného začiatku stal identifikátor. Znak `R` tu nemá osobitný význam, slúži iba ako záchranný znak, ktorý umožní sa automatu jednoznačne posunúť do ďalšieho stavu.

8.2 Implementácia

Lexikálna analýza bola implementovaná v jazyku TypeScript v súbore `src/lib/lexer.ts`. Implementácia modeluje konečný stavový automat, ktorý postupne prechádza medzi stavmi podľa vstupného znaku a po dosiahnutí akceptačného stavu vytvorí príslušný token.

Token sa potvrzuje až po načítaní jedného ďalšieho znaku. Tento dodatočný znak slúži ako overenie, že ide naozaj o daný token a následne sa spracuje znova v stave `Q0`, aby sa nestratil. Napríklad pri postupnosti `READ(` sa token `READ` potvrdí až znakom `(`, ktorý sa potom spracuje ešte raz. Vďaka tomu nie je potrebné znak vracať späť do vstupu, ale automat sa pri potvrdení tokenu posunie o dva kroky.

8.3 Testovacie scenáre

Testovanie lexikálnej analýzy bolo rozdelené na viacero vstupných súborov, aby bolo možné pokryť bežné prípady aj hraničné situácie. Súbor `public/examples/simple-example.smol` slúži ako jednoduchý pozitívny príklad, zatiaľ čo `public/examples/complex-example.smol` obsahuje komplexný vstup, ktorý pokrýva parsovanie všetkých tokenov.

Pre chybové stavy bol pripravený súbor `public/examples/errors/lex-error-recovery.smol`, ktorý obsahuje príklady chýb lexikálnej analýzy a overuje správanie pri zotavení z chyby.

9 Syntaktická analýza

Syntaktická analýza slúži na overenie, či postupnosť tokenov vytvorená lexikálnym analyzátorom zodpovedá `LL(1)` gramatike jazyka SMALL. Parser je realizovaný ako prediktívny analyzátor riadený prechodovou tabuľkou. Pri spracovaní vstupu využíva zásobník symbolov, vstupný buffer tokenov a postupne buduje syntaktický strom.
Hlavnou úlohou syntaktickej analýzy je:

- rozhodnúť o syntaktickej správnosti vstupu,

- aplikovať pravidlá podľa aktuálneho neterminálu a vstupného tokenu,
- vytvárať syntaktický strom,
- zaznamenávať priebeh analýzy,
- v obmedzenej miere pokračovať aj po výskyte chyby.

9.1 Parser

Parser pracuje štandardným spôsobom pre `LL(1)` analýzu. Na začiatku je na zásobník vložený štartovací symbol gramatiky a koncový symbol `$`. Počas spracovania parser vždy porovnáva vrchol zásobníka s aktuálnym vstupným tokenom.

- Ak je na vrchole terminál a zodpovedá aktuálnemu tokenu, parser ho spáruje a pokračuje ďalej.
- Ak je na vrchole neterminál, parser podľa prechodovej tabuľky vyberie vhodné pravidlo a nahradí ním neterminál na zásobníku.
- Ak sa použije ε-pravidlo, parser vstup neposúva.
- Ak sa na zásobníku aj na vstupe nachádza koncový symbol, vstup je prijatý.

Keďže ide o `LL(1)` parser, pri rozhodovaní stačí vždy jeden vstupný token dopredu a nie je potrebný *back-tracking*.

9.2 Syntaktický strom

Počas analýzy sa zároveň vytvára syntaktický strom (obrázok č. 1). Každý použitý symbol gramatiky má svoj uzol a pri aplikovaní pravidla sa vytvárajú jeho potomkovia podľa pravej strany produkcie. Pri úspešnom spárovaní terminálu sa do uzla uloží konkrétny token zo vstupu.
Takto vytvorený strom umožňuje lepšie pochopiť priebeh derivácie a zároveň vizualizovať výsledok syntaktickej analýzy.

9.3 Priebeh analýzy a logovanie

Parser si počas spracovania ukladá jednotlivé kroky analýzy (obrázok č. 2). V každom kroku sa zaznamenáva aktuálny stav zásobníka, pozícia vo vstupe, zvolená akcia a aktuálny stav syntaktického stromu. Vďaka tomu je možné zobraziť priebeh parsera krok po kroku.
Zaznamenávajú sa najmä tieto typy udalostí:

- inicializácia parsera,
- aplikovanie pravidla,
- spárovanie terminálu,
- prijatie vstupu,
- chyba a prípadné zotavenie z chyby.

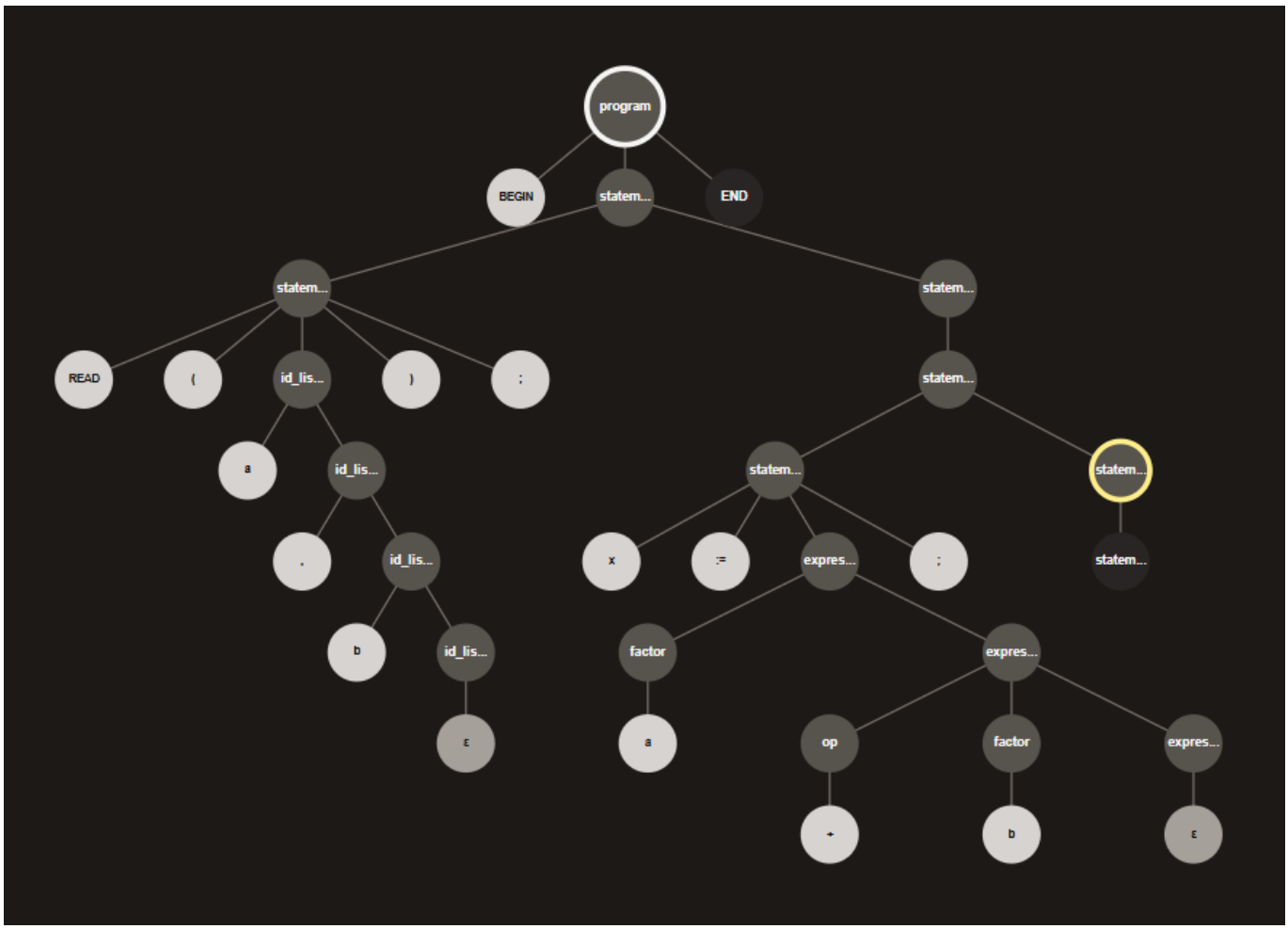

Figure 1: Ukážka syntaktického stromu pre vstupný program.

9.4 Zotavenie z chýb

Súčasťou parsera je aj jednoduché zotavenie z chýb. Okrem režimu, v ktorom sa analýza po prvej chybe ukončí, boli implementované aj dve základné stratégie:

- `skip-until-found` – parser preskakuje tokeny, kým nenájde vhodný bod na pokračovanie,
- `add-missing` – parser doplní chýbajúci token a pokračuje ďalej.

```
1  [INIT] Parser initialized. Starting symbol: program
2  [EXPAND] program -> BEGIN statement_list END (rule 1, lookahead: BEGIN)
3  [MATCH] BEGIN = "BEGIN" at line 1:0-4
4  [EXPAND] statement_list -> statement statement_list' (rule 2, lookahead: READ)
5  [EXPAND] statement -> READ LPAREN id_list RPAREN SEMI (rule 6, lookahead: READ)
6  [MATCH] READ = "READ" at line 2:0-3
7  [MATCH] LPAREN = "(" at line 2:4
8  [EXPAND] id_list -> IDENT id_list' (rule 11, lookahead: IDENT)
9  [MATCH] IDENT = "a" at line 2:5
10 [EXPAND] id_list' -> COMMA id_list (rule 13, lookahead: COMMA)
```

Figure 2: Ukážka záznamu krokov syntaktickej analýzy.

Cieľom týchto stratégií je umožniť pokračovanie analýzy aj pri chybnom vstupe, aby používateľ dostal viac informácií než len prvú nájdenú chybu. Ak sa parseru podarí po chybe pokračovať, výsledok je označený ako `correct-with-errors`.

9.5 Implementácia

Syntaktická analýza bola implementovaná v jazyku TypeScript v triede `SyntaxParser`. Implementácia využíva prechodovú tabuľku vytvorenú z tokenizovanej `LL(1)` gramatiky a pri spracovaní vstupu buduje syntaktický strom aj záznam jednotlivých krokov analýzy.
Z dôvodu bezpečnosti obsahuje parser aj limit maximálneho počtu krokov, aby sa predišlo nekonečnému cyklu pri nesprávnom vstupe alebo chybnej konfigurácii (pri testovaní nám stačil limit `5000` krokov).

9.6 Testovanie

Funkčnosť syntaktickej analýzy bola overovaná dvoma spôsobmi. Prvým bolo testovanie na vstupnom súbore, ktorý obsahuje konštrukcie pokrývajúce všetky pravidlá gramatiky, čím sa overila správnosť parsera na úplnom jazyku.
Druhým bolo automatizované testovanie pomocou súboru `syntax_parser_test.ts`, v ktorom bol pripravený test pre každé pravidlo gramatiky aj pre jednotlivé stratégie zotavenia z chýb. Overovalo sa najmä správne aplikovanie pravidiel, korektné prijatie vstupu a správne správanie parsera pri chybových stavoch.

10 Ako spustiť

Program je možné spustiť pomocou `python -m http.server 42069 --directory dist` a následne ho otvoriť v prehliadači na adrese `http://localhost:42069`. Keďže ide o predpripravený build, nie je potrebné nič ďalšie kompilovať.

Najprv sa vyberie vstupný súbor z pripravených príkladov v `public/examples/`. Na úvodnej obrazovke sa otvorí výber súboru, v ktorom sú oddelené korektné príklady a príklady s chybou. Po zvolení vstupu nasleduje lexikálna konfigurácia, kde sa vyberie režim zotavenia z chýb, alebo sa ponechá vypnutý. V nasledúcom lexikálnom kroku je možné postupne prechádzať vstupným textom, sledovať buffer znakov, emitované tokeny a logy lexeru.

Po dokončení lexikálnej analýzy sa pokračuje na syntaktickú konfiguráciu, kde sa opäť vyberie režim zotavenia z chýb. V syntaktickom kroku sa zobrazuje postupné spracovanie tokenov, zásobník, syntaktický strom a logy parsera. Po prejdení celého vstupu sa zobrazí obrazovka s výsledkami a súhrnom oboch fáz.

11 Rozdelenie práce

Lukáš Častven sa venoval odvodeniam z BNF do `LL(1)` gramatiky a implementácii lexikálnej analýzy. Michal Kilian sa venoval frontendu a syntaktickej analýze. Rozdelenie cca 50/50.